

Fragmento 1 — Script de despliegue

#	Problema Detectado	Severidad	Impacto Real en la Operación
1	Contraseñas escritas en texto plano en el script DB_PASSWORD, JWT_SECRET	 Crítico	Cualquier persona o desarrollador que lea este archivo tiene acceso total a la base de datos de producción y a clonar tokens de autenticación de usuarios.
2	Falta de detención ante errores Sin set -e	 Crítico	Si git pull o npm install fallan, el script no se detiene; intenta reiniciar el servicio con código incompleto o corrupto.
3	Permisos excesivos en carpeta de archivos chmod 777 uploads	 Crítico	Permite que cualquier usuario del sistema lea, modifique o ejecute archivos maliciosos dentro del servidor.
4	Respaldo de BD después del despliegue	 Alto	Si la nueva versión corrompe la base de datos durante el inicio o migraciones, el respaldo guardará los datos ya corruptos. El respaldo debe ser antes de actualizar.
5	Uso inseguro de scp e IP fija en red local 192.168.1.10	 Alto	Si la IP cambia o la conexión falla, la copia del respaldo no se realiza sin dar alerta. Además, requiere credenciales expuestas para conectarse.
6	Falta de verificación de la instalación de dependencias Uso de npm install	 Medio	En producción se debe usar npm ci para instalar exactamente las versiones fijadas en package-lock.json. npm install puede instalar librerías nuevas sin probar.
7	Mensaje de éxito engañoso echo "Deploy completado"	 Medio	Imprime que todo terminó bien sin importar si la aplicación realmente arrancó o falló.

Fragmento 2 — Workflow de GitHub Actions

#	Problema Detectado	Severidad	Impacto Real en la Operación
1	Contraseña SSH expuesta en el código <pre>sshpass -p 'servidor123'</pre>	 Crítico	La contraseña del servidor de producción está visible en texto plano dentro del repositorio, permitiendo el acceso total a usuarios no autorizados.
2	Conexión como usuario root a IP privada <pre>root@10.0.0.5</pre>	 Crítico	Desplegar con el usuario root otorga privilegios ilimitados en el servidor si la sesión es comprometida. Además, los <i>runners</i> públicos de GitHub no pueden conectarse a una IP privada 10.0.0.5 sin una VPN o Runner Autofirmado/Local.
3	Despliegues automáticos desde ramas no estables <pre>branches: [main, develop, feature/*]</pre>	 Crítico	Cualquier commit en una rama de característica (feature/*) desencadena un despliegue directo a producción, destruyendo la estabilidad del entorno.
4	Falta de pruebas automatizadas obligatorias <pre>No existe npm test</pre>	 Alto	El código se compila y se despliega directamente sin verificar la lógica de negocio mediante pruebas unitarias o de integración.
5	Reinicio masivo e indiscriminado de servicios <pre>pm2 restart all</pre>	 Alto	Reiniciar todos los procesos de PM2 en lugar del servicio específico puede tirar aplicaciones secundarias no relacionadas que corren en el mismo servidor.
6	Sin estrategia de Rollback ni aislamiento <pre>git pull directo en producción</pre>	 Medio	Hacer git pull en producción mezcla cambios en vivo. Si el build del servidor falla a la mitad, no hay forma automatizada de volver a la versión anterior (rollback).

#	Problema Detectado	Severidad	Impacto Real en la Operación
7	Lógica de notificación de estado errónea Uso de if: always() con mensaje estático	● Medio	La notificación en Slack envía siempre el texto "Deploy exitoso", incluso si el build, la instalación o la conexión SSH fallan

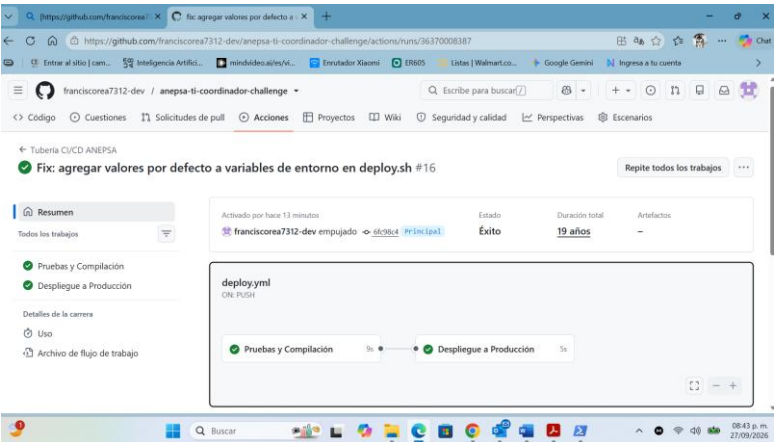
¿Por qué el pipeline dice "Deploy exitoso" incluso cuando el build o el deploy fallan?

Es a causa de esta línea, la cual indica a GitHub Actions que fuerce la ejecución de ese paso sin importar el resultado de los pasos anteriores

if: always()

Propuesta de Workflow:

<https://github.com/franciscorea7312-dev/anepsa-ti-coordinador-challenge>



Fragmento 3 — Configuración de Nginx

Lista todos los problemas con severidad

Severidad	Componente Afectado	Descripción Técnica del Problema	Impacto de Seguridad / Operativo
 CRÍTICA	Repository / Git	Fuga de Credenciales en el Historial de Git (.env commiteado)	Las variables de entorno de producción (llaves de API, contraseñas de BD, secretos JWT) están visibles en el historial de commits. Cualquiera con acceso al repositorio puede clonarlo o consultar el historial para extraerlas.
 CRÍTICA	Server / Nginx	Exposición Directa de Archivos Ocultos a Nivel Web (nginx.conf)	La configuración de Nginx no bloquea peticiones a archivos que inicien con .. Un atacante puede descargar directamente el archivo .env mediante una petición HTTP (http://dominio.com/.env).
 ALTA	Repository / Git	Inexistencia de Exclusión en .gitignore	El archivo .gitignore no incluye la regla .env, permitiendo que futuros cambios o nuevos archivos de credenciales locales se suban accidentalmente al servidor remoto.
 ALTA	Server / Nginx	Ausencia de Cabeceras de Seguridad (Security Headers) en Nginx	No existen reglas para enviar cabeceras como X-Frame-Options, X-Content-Type-Options o Content-Security-Policy (CSP), dejando la aplicación vulnerable a ataques como Clickjacking y XSS.
 MEDIA	Server / Nginx	Falta de Redirección Forzada a HTTPS (SSL/TLS)	El bloque del servidor escucha en el puerto HTTP (80) sin una regla de redirección obligatoria (HTTP 301/302) hacia el puerto seguro HTTPS (443).

Con quitarlo del último commit es suficiente? Justifica técnicamente

Aspecto	Evaluación / Diagnóstico	Explicación Técnica y Riesgo Operativo
¿Es suficiente eliminarlo del último commit?	✗ NO	Eliminar el archivo con un commit posterior (<code>git rm .env</code>) solo lo remueve del estado actual (HEAD), pero permanece intacto en la historia de commits anteriores.
Inmutabilidad del Historial de Git	● CRÍTICO	Git almacena los cambios como un grafo acíclico dirigido (DAG) de snapshots inmutables. El archivo <code>.env</code> continúa existiendo en los objetos <i>blob</i> de cada commit realizado durante los últimos 8 meses.
Riesgo de Extracción Automatizada	● CRÍTICO	Cualquier usuario o bot de escaneo (ej. GitGuardian, TruffleHog) puede clonar el repositorio y recuperar las credenciales completas ejecutando un simple comando: <code>git log -p -- .env</code> .
Alcance y Exposición Remota	● ALTO	Tras 8 meses expuesto, las credenciales ya residen en forks, clones locales de desarrolladores, servidores de integración continua (CI/CD) y cachés remotos de GitHub.
Acción Inmediata Obligatoria 1	🔑 INVALIDACIÓN	Rotación e invalidación de credenciales: Asumir compromiso total y revocar/cambiar de inmediato todas las llaves de API, contraseñas de BD y secretos JWT.
Acción Inmediata Obligatoria 2	🔪 PURGA	Purga del historial: Reescribir la historia del repositorio eliminando el objeto <code>.env</code> de todos los commits mediante <code>bfg --delete-files .env</code> o <code>git filter-repo</code> .
Acción Inmediata Obligatoria 3	🔄 REESCRITURA	Forzado de ramas: Ejecutar un <i>force push</i> (<code>git push origin --force --all</code>) para sobrescribir la historia en los servidores remotos.

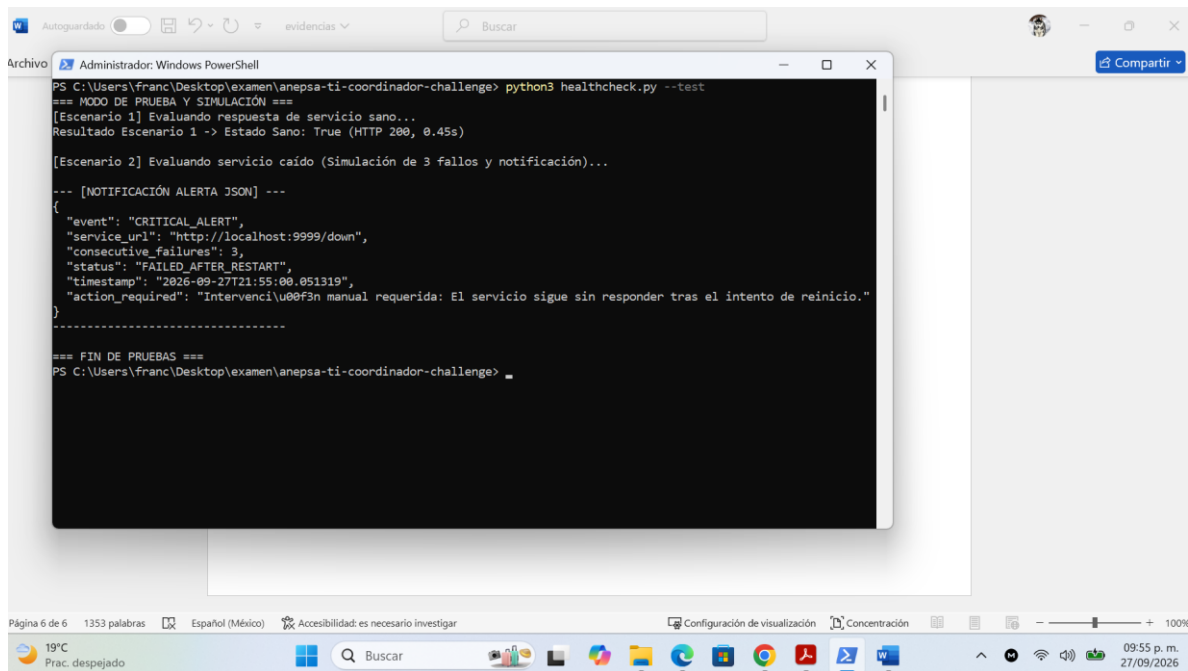
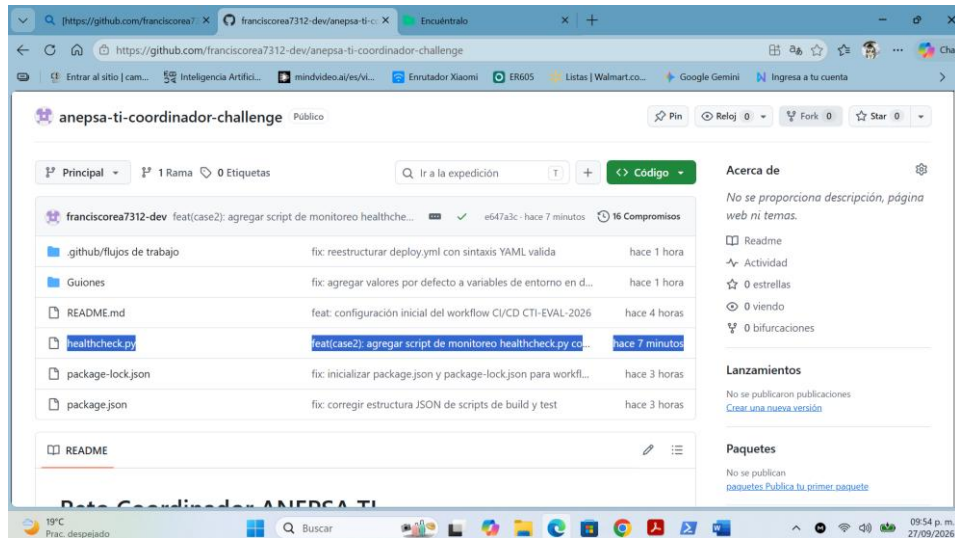
plan de remediación completo

Fase	Control / Acción	Descripción Técnica
1. Rotación e Invalidación Inmediata	🔑 Revocación y Cambio de Credenciales	Revocar y regenerar de inmediato todas las credenciales expuestas en <code>.env</code> (contraseñas de BD, JWT secrets y claves API como <code>API_KEY_BANCO</code>). Actualizar los entornos de producción con los nuevos secretos.
2. Purga del Historial de Git	🔪 Eliminación Permanente con git filter-repo / BFG	Purgar el archivo <code>.env</code> de todo el historial de commits y tags del repositorio mediante <code>bfg --delete-files .env</code> o <code>git filter-repo --invert-paths --path .env</code> . Ejecutar <code>git push origin --force --all</code> para sobrescribir las ramas remotas.

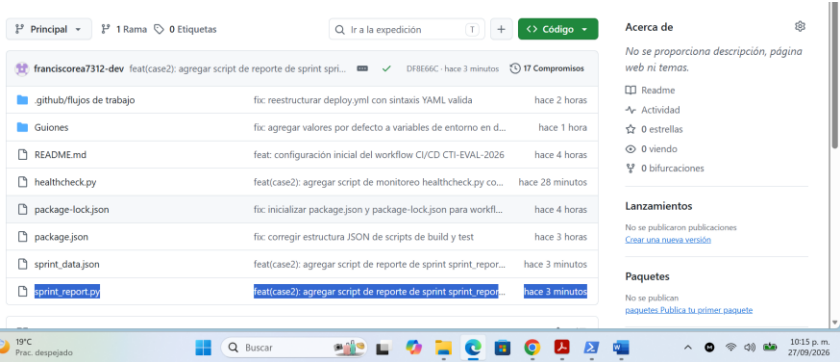
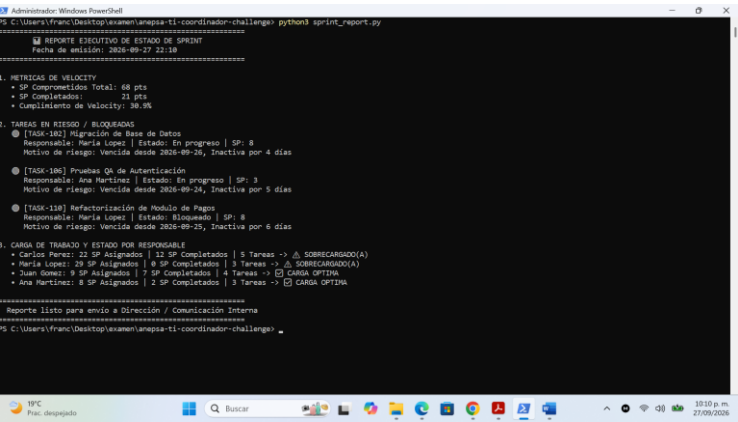
Fase	Control / Acción	Descripción Técnica
3. Configuración del Servidor	 Hardening en Nginx (nginx.conf)	Agregar una regla explícita de bloqueo en Nginx para denegar el acceso público a archivos ocultos (location ~ /\. { deny all; return 404; }), previniendo la descarga directa de archivos de configuración por HTTP.
4. Prevención Futura (Control 1)	 Exclusión Estricta en .gitignore	Agregar .env, .env.* y archivos de configuración sensibles a .gitignore. Proporcionar un archivo de plantilla seguro llamado .env.example en el repositorio sin credenciales reales.
5. Prevención Futura (Control 2)	 Hooks Locales Pre-commit (Gitleaks / Husky)	Implementar un hook de pre-commit utilizando herramientas como Gitleaks o TruffleHog para escanear localmente los archivos en <i>staged</i> antes de permitir la creación de un commit.
6. Prevención Futura (Control 3)	 Escaneo Automático en CI/CD (Secret Scanning)	Integrar una etapa de análisis de secretos en el pipeline de GitHub Actions (.github/workflows/deploy.yml) usando GitGuardian o Gitleaks Action para abortar la compilación si se intenta subir un secreto.
7. Prevención Futura (Control 4)	 Gestor Centralizado de Secretos (Vault / Cloud Secrets)	Desacoplar completamente los archivos de entorno locales del servidor e integrar un gestor de secretos centralizado (como HashiCorp Vault, AWS Secrets Manager o GitHub Environment Secrets) para inyectar variables en tiempo de ejecución de manera cifrada y auditable.

Caso 2

Ejercicio A — Script de monitoreo y autorrecuperación

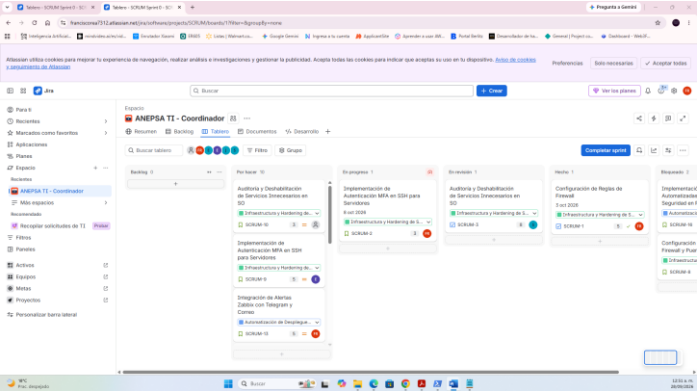


Ejercicio B — Script de reporte de estado de sprint

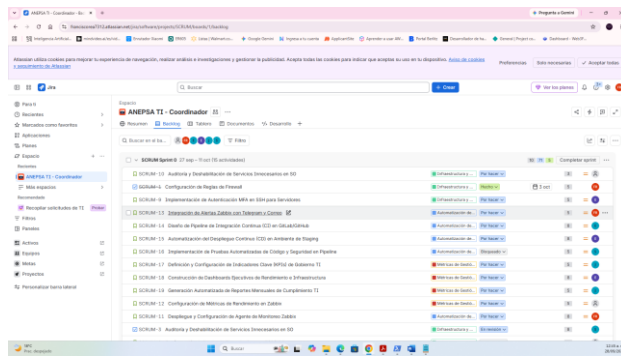


Ejercicio C Tablero real de gestión

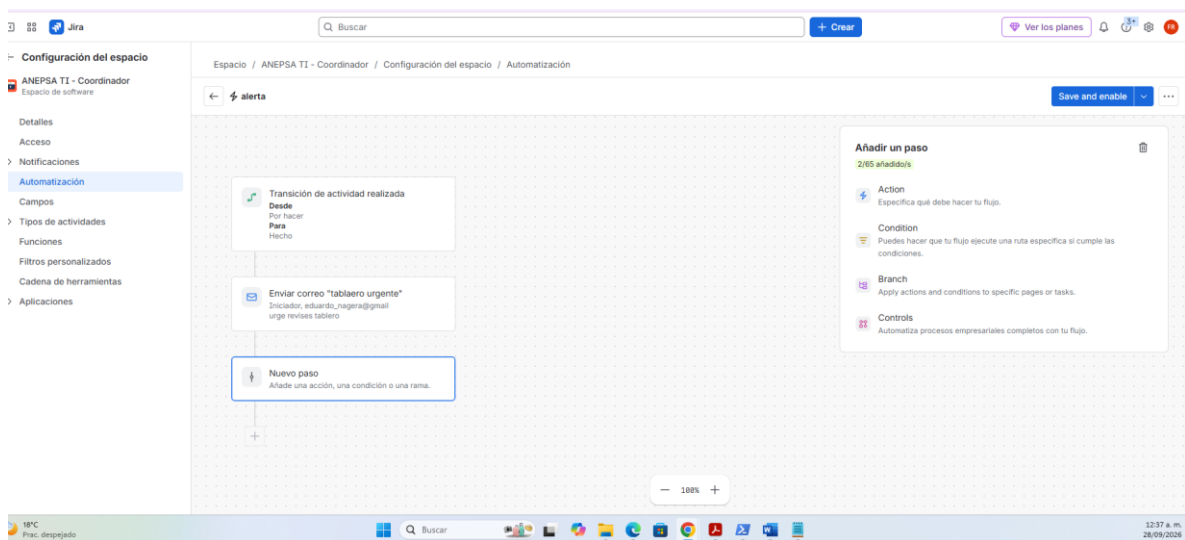
Estructura de épicas/historias/story points coherente



Sprint activo con asignaciones reales

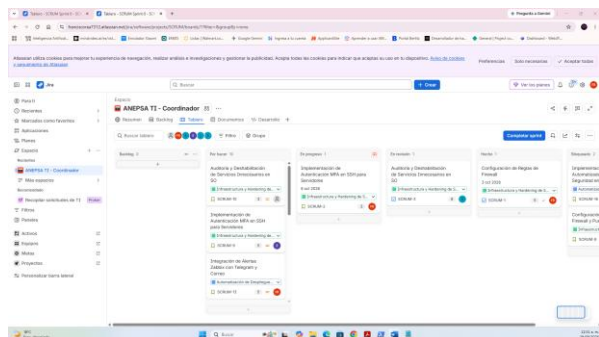


3. Automatización o WIP limit configurado y funcional



Enlace accesible + evidencia de fecha

<https://franciscorea7312.atlassian.net/jira/software/projects/SCRUM/boards/1?filter=&groupBy=None>



CASO 3 - GESTIÓN DE EQUIPO Y LIDERAZGO

3.1 Conflicto técnico entre dos colaboradores

. Plan de Acción en las Primeras 48 Horas

- Conversación inicial con Georgina López:
 - **Objetivo:** Validar su compromiso con la calidad y encauzar su rigor técnico sin generar desgaste emocional y que se puedan sentir escuchado.
 - **Guión aproximado:**

"Hola Georgina. He notado en los dailies y code reviews cierta tensión y desgaste emocional en el equipo. Valoro profundamente tu rigor y compromiso con la calidad para proteger al producto de errores, pero quiero asegurarme de que esta carga de revisión no te esté generando un estrés innecesario. ¿Cómo te estás sintiendo tú con este proceso? Me gustaría que encontremos juntos una forma de cuidar la excelencia técnica sin que esto afecte el ánimo del equipo y ni a ti. En que te puedo apoyar, quieres que revisemos algo en especial para platicarlo y exponerlo. "

- Conversación inicial con Eduardo Zurita:
 - **Objetivo:** Ofrecer un espacio seguro de escucha para identificar si la presión o la falta de claridad están afectando sus entregas.
 - **Guión aproximado:**

"Hola Eduardo. Quería platicar contigo en privado porque he visto que las revisiones recientes han sido pesadas y sé que eso puede ser muy frustrante en el día a día. Quiero escucharte: ¿sientes que las guías actuales son claras, o la presión por sacar los tickets a tiempo te está abrumando? Mi prioridad es apoyarte y ayudarte, no señalarte. Vamos a resolver esto juntos. En que tienes dudas "

- Seguimiento a las 24 horas (Check-in rápido):
 - **Objetivo:** Validar el estado emocional de ambos tras la primera plática y asegurar que las tensiones no hayan escalado.
 - **Abordaje:**

"Hola [Georgina / Habarán], como estás. Solo quería pasar a saludarte y saber cómo te sientes hoy con el flujo de trabajo si tienes alguna sugerencia. Mi puerta sigue abierta para cualquier cosa que necesites, con gusto la platicamos y resolvemos "

2. Diagnóstico: ¿Problema de Proceso o Interpersonal?

- Cómo distinguirlo:
 - Es un problema de proceso si el equipo opera en la ambigüedad (criterios de calidad subjetivos o no escritos), lo que genera que los desarrolladores debatan criterios personales en cada *code review*.
 - Es un problema interpersonal si la tensión traspasa lo técnico y se convierte en juicios sobre la capacidad del compañero o defensas emocionales cerradas, hacen ataques directos ala otra persona
- Preguntas de diagnóstico para cada uno:
 - A Georgina (la revisora): *"¿Sientes que el equipo te apoya cuando intentas mantener los estándares, o sientes que estás cargando sola con la responsabilidad de la calidad y tus compañeros lo toman a mal ?"*
 - A Eduardo (el autor): *"¿Qué emoción te queda cuando recibes los comentarios en tus pull requests? ¿Sientes que se evalúa tu código o que se cuestiona tu capacidad, por temas de calidad o reglas establecidas?"*

3. Llamado a la Colaboración, Cierre del Proyecto y Solución Estructural

- Invitación temporal para enfocar energías:
 - *"Equipo, sé que traemos diferencias de criterio técnico, pero tenemos un proyecto clave en puerta que requiere de todo nuestro talento unido. Los invito a que saquemos este compromiso juntos esta semana, apoyándonos mutuamente, y en cuanto terminemos, nos sentaremos los tres a buscar una solución definitiva."*

- Sesión conjunta de los tres (A los 3 días / Cierre del entregable):
 - Espacio de diálogo neutral donde ambos exponen sus puntos sin juicios, moderados por la coordinación.
- Solución Estructural Acordada:
 1. Automatización de guías de estilo: Integración de un linter automático en el repositorio para que la herramienta evalúe la sintaxis de forma neutral, eliminando la fricción personal en los *code reviews*.
 2. Definición clara de acuerdos en el Code Review: Establecer como equipo qué es un bloqueo crítico (*bugs*, seguridad) frente a una sugerencia opcional, evitando que las opiniones subjetivas frenen el flujo.
 3. Dinámicas de entendimiento mutuo: Espacios cortos de programación en pareja para alinear expectativas técnicas y fortalecer la confianza interpersonal.
 4. Hacer que estemos en contexto que somos equipo y debemos apoyarnos y yo no señalo simplemente escucho y trato de buscar las mejores soluciones para la empresa y ellos mismos.

3.2 Evaluación de desempeño técnico

Para cada perfil: mínimo 5 dimensiones evaluables (técnicas y de comportamiento), con escala de 1 a 5 y ejemplo concreto de qué diferencia un 3 de un 5.

Perfil	Dimensión Evaluable	Nivel 3 (Desempeño Estándar)	Nivel 5 (Desempeño de Excelencia)
Desarrollador Full Stack	1. Calidad de Código y Arquitectura	Escribe código funcional que pasa pruebas, pero requiere ajustes frecuentes en estilo o refactorización menor en los <i>code reviews</i> .	Diseña componentes altamente modulares y limpios; código autocontenido con cobertura de pruebas >85% y mínimos ajustes requeridos.
Desarrollador Full Stack	2. Velocidad y Entrega (Velocity)	Entrega historias dentro del sprint, aunque ocasionalmente subestima la complejidad de tareas nuevas.	Estima con alta precisión, identifica bloqueantes de forma temprana y acelera la entrega de valor sin sacrificar calidad técnica.
Desarrollador Full Stack	3. Colaboración y Code Reviews	Participa en revisiones de forma básica aprobando PRs, limitando sus comentarios a observaciones estéticas.	Proporciona retroalimentación constructiva, técnica y empática; actúa como mentor activo de perfiles junior.
Desarrollador Full Stack	4. Resiliencia ante la Presión	Mantiene la compostura ante cambios, aunque puede llegar a mostrarse abrumado si se acumulan múltiples tareas urgentes.	Gestiona con serenidad escenarios críticos o de alta presión, manteniendo comunicación clara y enfocada en soluciones.
Desarrollador Full Stack	5. Autonomía y Resolución	Resuelve tareas asignadas consultando frecuentemente al líder técnico o investigando dudas bloqueantes.	Investiga de forma independiente, propone soluciones arquitectónicas viables y documenta hallazgos para el equipo.

Perfil	Dimensión Evaluable	Nivel 3 (Desempeño Estándar)	Nivel 5 (Desempeño de Excelencia)
Automatización / QA	1. Robustez de Flujos / Pruebas	Crea scripts o flujos funcionales que cubren el camino feliz (<i>happy path</i>), pero fallan ocasionalmente ante excepciones.	Desarrolla flujos robustos con manejo integral de errores, reintentos automáticos, alertas preventivas y cobertura de casos extremos.
Automatización / QA	2. Detección Temprana (Shift-Left)	Ejecuta pruebas o validaciones cuando el desarrollo está avanzado o próximo a integrarse en staging.	Colabora desde la definición de requerimientos para anticipar fallas, integrando validaciones automáticas en etapas tempranas.
Automatización / QA	3. Mantenibilidad y Documentación	Mantiene los flujos operativos, aunque la documentación interna suele actualizarse de manera tardía o superficial.	Documenta exhaustivamente cada automatización (diagramas, variables, dependencias) facilitando su operación por terceros.
Automatización / QA	4. Optimización de Tiempos	Identifica y automatiza tareas manuales cuando se le solicita explícitamente desde la coordinación.	Audita proactivamente los procesos del área, proponiendo y ejecutando automatizaciones que reducen horas-hombre.
Automatización / QA	5. Trabajo en Equipo y Soporte	Resuelve incidencias de automatización cuando se le reportan, manteniendo canales de comunicación estándar.	Se anticipa a las fallas mediante tableros de monitoreo y apoya solidariamente al equipo de desarrollo ante bloqueos críticos.
Soporte / Operaciones	1. Tiempo de Respuesta (SLA)	Atiende y resuelve incidentes dentro de los tiempos estándar establecidos, escalando adecuadamente los casos complejos.	Optimiza los tiempos de resolución (MTTR), identificando patrones recurrentes para evitar que los incidentes se repitan.
Soporte / Operaciones	2. Monitoreo y Proactividad	Monitorea los sistemas utilizando las herramientas preconfiguradas y responde a alertas automáticas.	Configura métricas y alertas tempranas predictivas, detectando anomalías y mitigando riesgos antes de impactar la operación.
Soporte / Operaciones	3. Documentación de Incidentes	Registra los tickets con la información básica del incidente y su solución temporal o definitiva.	Elabora bitácoras detalladas y análisis de causa raíz (RCA), enriqueciendo la base de conocimiento del área.
Soporte / Operaciones	4. Comunicación con Stakeholders	Comunica el estado de las incidencias de forma clara cuando se le solicita un reporte de estatus.	Mantiene informados a los afectados con empatía, claridad y tiempos estimados precisos, generando confianza en crisis.
Soporte / Operaciones	5. Gestión de Carga Operativa	Ejecuta operaciones rutinarias programadas (cortes, respaldos, despliegues) siguiendo manuales establecidos.	Automatiza tareas repetitivas de su propia operación diaria, liberando tiempo para apoyar proyectos de mejora continua.

¿Cómo evaluar objetivamente a alguien cuyo trabajo es "invisible" cuando todo funciona bien?

Para evitar la trampa de evaluar únicamente a quienes "hacen mucho ruido" o resuelven incendios visibles, se utilizan métricas de confiabilidad proactiva y prevención:

- Indicadores de Estabilidad (SLIs / SLOs): Medir el porcentaje de tiempo de actividad (*uptime*), la tasa de fallos evitada y la reducción neta de horas manuales invertidas por la empresa.
- Auditoría de Salud Preventiva: Evaluar la calidad de la documentación técnica, la ausencia de deuda técnica en sus módulos y la frecuencia con la que actualiza o mejora sus flujos sin que se lo tengan que pedir.
- Revisiones de Arquitectura (Peer Review): Evaluar el diseño de sus sistemas o scripts analizando su resiliencia ante fallos externos, escalabilidad y limpieza estructural.

¿Qué hacer si el resultado de la evaluación es negativo pero la persona es la única que conoce un sistema crítico?

Esta es una vulnerabilidad operativa grave

Separar el rendimiento del plan de mitigación técnica: Nunca usar el conocimiento de un sistema crítico como escudo para tolerar un desempeño negativo o actitudes tóxicas; sin embargo, tampoco se pueden tomar decisiones drásticas sin antes asegurar la continuidad del negocio.

Plan de Mitigación y Transferencia de Conocimiento Forzoso (Acción Inmediata):

- Shadowing intensivo: Asignar de inmediato a un segundo colaborador (por ejemplo, del equipo de desarrollo o soporte) para que funcione como sombra (*shadow*) en todas las operaciones, despliegues y mantenimientos de ese sistema crítico.
- Ingeniería inversa y documentación obligatoria: Establecer entregables de documentación técnica estricta (diagramas de arquitectura, manuales de operación, variables de entorno, mapas de dependencias) en un plazo no mayor a 2 semanas.
- Validación cruzada: Realizar un simulacro de fallo o intervención en un ambiente controlado donde el colaborador en cuestión no participe, comprobando que el resto del equipo ya es capaz de operar y entender el sistema.

3.3 Riesgo de fuga de talento clave

Qué conversación tienes con esta persona en los próximos 3 días?

La conversación debe darse en un entorno de absoluta confianza, un café fuera de la oficina o un espacio privado donde Esteban se sienta seguro, valorado y escuchado sin temor a represalias.

- Guión de apertura con sensibilidad:

"Hola Esteban. Quería invitarte a platicar un momento a solas. Sé que cargas con una parte fundamental de nuestra operación con los flujos de N8N, y valoro enormemente tu talento y dedicación de todos estos meses. Me he enterado de manera informal que estás explorando nuevas oportunidades profesionales. Antes que cualquier indicador de proyecto, quiero decirte que tu bienestar y tu crecimiento me importan genuinamente. No estoy aquí para reclamarte nada, sino para escucharte de corazón: ¿cómo te estás sintiendo en el equipo y qué estás buscando para tu futuro?"

- Las 3 preguntas:
 1. *"¿Qué aspectos de tu día a día, de los proyectos o de tu desarrollo personal sientes que se han apagado o ya no te motivan a dar lo mejor de ti?"*
 2. *"¿Hay algo en la dinámica del equipo, en la carga de trabajo o en el soporte que te esté generando frustración y desgaste emocional que no hayamos sabido ver a tiempo?"*
 3. *"Si estuviera en mis manos transformar tu rol actual para que te sientas entusiasmado, valorado y con ganas de seguir creciendo con nosotros, ¿qué tendría que cambiar a partir de esta semana?"*

2. Plan de mitigación de riesgo técnico en paralelo

Para cuidar la operación de los 15 flujos críticos de N8N justificaremos las acciones bajo una iniciativa empática de "Bienestar y Cuidado Mutuo del Equipo":

- Justificación hacia el equipo: Anunciar que, para evitar que las vacaciones, el cansancio o los imprevistos de una sola persona detengan la operación, implementaremos un modelo de co-responsabilidad y respaldo mutuo en áreas críticas.
- Documentación natural y respetuosa: Asignar a un segundo compañero para que colabore de la mano con Esteban (*shadowing*) en los flujos de N8N, documentando paso a paso la arquitectura y las dependencias, presentándolo como una oportunidad para que el equipo crezca y aprenda de su experiencia.
- Resguardo preventivo: Asegurar que los respaldos y versiones de los flujos queden respaldados en el control de versiones corporativo como parte de las buenas prácticas generales de seguridad.

Contra-oferta: ¿Bajo qué condiciones ofrecerla y bajo cuáles no?

- ¿Bajo qué condiciones SÍ ofrecer una contra-oferta?
 - Cuando el motivo principal por el cual busca salir es estrictamente económico, y existe un compromiso real y humano de su parte por quedarse, ayudando a consolidar el área y transmitiendo su conocimiento a los demás.
 - Cuando su liderazgo técnico es irremplazable a corto plazo y la contra-oferta viene acompañada de un plan claro de desarrollo.
- ¿Bajo qué condiciones NO ofrecer una contra-oferta?
 - Cuando el trasfondo de su salida es un ambiente que ya no le aporta a su vida personal y tiene problemas
 - Si la contra-oferta rompe la equidad salarial interna con el resto del equipo de TI.
 - Si está usando la oferta de otra empresa únicamente como un recurso de presión para negociar sin un deseo real de permanencia.

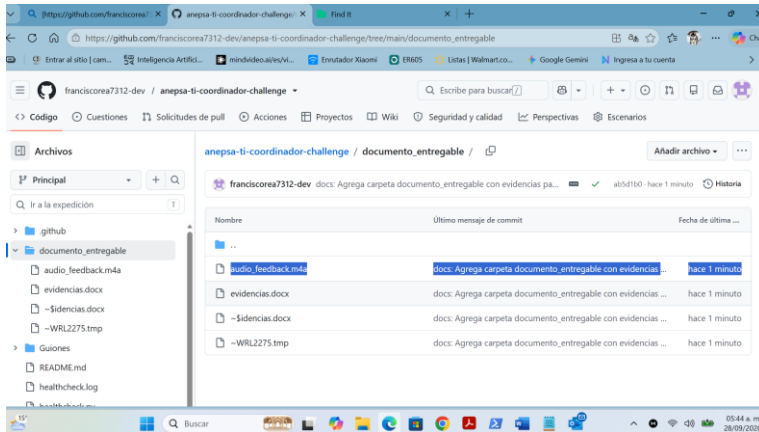
- Tú: "Hola Carlos, gracias por darme unos minutos. Quería invitarte a platicar un momento a solas. Sé que estás metido de lleno en los tickets del sprint y que le pones mucho esfuerzo a cada desarrollo. ¿Cómo te has sentido esta semana con la carga de trabajo y el ritmo general?"
- Carlos "Hola. Pues ahí voy, sacando los pendientes, aunque la verdad siento que las estimaciones que me piden siempre son imposibles y me ponen la vara muy alta sin conocer lo que tardan los detalles técnicos."
- Tú: "Te entiendo perfectamente, Tienes toda la razón en que los detalles técnicos a veces ocultan complejidades que no se ven a simple vista, y mi intención no es presionarte Al contrario, me preocupa que te estés llevando más tiempo del planeado (un 30-40% extra) y quiero entender qué obstáculos te están deteniendo para apoyarte."
- Carlos "Es que no es por mí, es que los requerimientos siempre cambian a la mitad o las dependencias con otros equipos y me retrasan,"
- Tú "Es un punto muy importante y sé que los cambios de última hora y las dependencias externas nos atrasan. Vamos a revisar eso juntos; también he notado que en tareas donde no hay cambios externos te estas tardando."
- Carlos: " ahora me van a decir que soy lento o que no rindo lo suficiente como los demás."

🔍 Intercambio 4:

- Tú "Quiero que sepas algo amigo: para mí no eres lento, eres un elemento valioso del equipo. Pero Prefiero mil veces una estimación realista y holgada a una fecha forzada que termine estresándote y afectando tu descanso. Aquí nadie te está juzgando por el tiempo, mi meta es que las métricas reflejen tu realidad para que trabajes con tranquilidad."
- Carlos "Es que... sí me pesa un poco ver que los demás cierran tickets más rápido y siento que siempre tengo que estar corriendo para estar a su nivel."
- Tú: "Te agradezco muchísimo que me compartas eso, porque requiere mucha honestidad reconocerlo. Es completamente normal que cada quien tenga su propio ritmo y estilo para trabajar. ¿Qué te parecería si en lugar de ver problemas vemos soluciones."
- Carlos: "Pues la verdad sí me ayudaría, porque a veces me atoro en la arquitectura o en resolver una excepción y me da pena decirlo en el daily para que no piensen que no sé."
- Tú : "Me da muchísima tranquilidad que me digas esto. Por favor, borra de tu mente la idea de que pedir ayuda es una debilidad; al contrario, es una muestra de profesionalismo. Si ves que una tarea no avanza como esperabas, búscame de inmediato y lo revisamos juntos."
- Carlos: "De acuerdo, lo tendré en cuenta. A veces uno se encierra en resolverlo solo por orgullo y miedo al que dirán."
- Tú "Es una reacción muy humana. Para este siguiente sprint, te propongo que estimemos con total libertad, sumándole un colchón de tiempo holgado para que trabajes sin esa presión. ¿Te parece bien si lo probamos así esta semana y lo revisamos en nuestro próximo uno a uno?"
- Carlos: "Sí, la verdad eso me quitaría un peso enorme de encima. Gracias por escucharme"
- Tú: "Al contrario, gracias a ti por la confianza. Valoro muchísimo tu talento y quiero que te sientas respaldado. Vamos a darle con todo a este proyecto, cuentas conmigo para lo que necesites."
- Carlos: "Perfecto, así lo haremos. Muchas gracias."

Entrega adicional: graba un audio o video

[anepsa-ti-coordinador-challenge/documento_entregable at main · franciscorea7312-dev/anepsa-ti-coordinador-challenge](https://github.com/franciscorea7312-dev/anepsa-ti-coordinador-challenge/tree/main/documento_entregable)



Caso 4 GESTIÓN DE PROYECTOS, JIRA Y GOBERNANZA

4.1 Priorización de backlog con recursos limitados

Cálculo de la Capacidad Real Disponible del Equipo por Semana

Cálculo de la Capacidad Real Disponible del Equipo por Semana

- **Composición del equipo:**
 - 4 Desarrolladores
 - 1 QA
 - 1 Especialista en automatización
 - Total de recursos: 6 personas.
- **Horas base:** Cada uno trabaja 40 horas a semana $\rightarrow 6 \times 40 = 240$ horas totales de capacidad bruta por semana.
- **Consumo en soporte:** La dirección indica que el 30% de la capacidad se consume en soporte operativo.
- **Capacidad real disponible para proyectos:**
 $\text{Capacidad Real} = 240h \times (1 - 0.30) = 168$ horas netas por semana dedicadas a nuevas iniciativas.

Método de Priorización Explícito (WSJF - Weighted Shortest Job First)

Utilizaremos el modelo WSJF (*Weighted Shortest Job First*), ideal para entornos ágiles con recursos limitados.

- **Cost of Delay (CoD)** = Valor (1 a 5) + Urgencia (1 a 5) + Dependencias/Riesgo (1 a 5). Para simplificar con los datos de la tabla, asignaremos valores numéricos estándar a las etiquetas: Alto = 3, Medio = 2, Bajo = 1.
- **Duración / Esfuerzo (Job Size)** = Horas estimadas.

WSJF = costo/JObsize x (se multiplica por un factor de escala para facilitar la lectura, por ejemplo 100)

ID	Iniciativa	Esfuerzo (h)	Valor (Alto=3)	Urgencia (Alto=3)	CoD (Valor + Urgencia)	WSJF (CoD / Esfuerzo x 100)
A	Automatizar conciliación manual	160	3 (Alto)	3 (Alto)	6	$\frac{6}{160} \times 100 = 3.75$
C	Migración de autenticación (Seguridad)	140	2 (Medio)	3 (Alto)	5	$\frac{5}{140} \times 100 = 3.57$
D	Dashboard de métricas para Dirección	90	2 (Medio)	2 (Medio)	4	$\frac{4}{90} \times 100 = 4.44$ (pero depende de secuenciación)
B	Portal de autoservicio para clientes	220	3 (Alto)	2 (Medio)	5	$\frac{5}{220} \times 100 = 2.27$ (Requiere C al 50%)

Respuesta a la Dirección

Director: Entiendo perfectamente la urgencia y el valor estratégico que representan estas 4 iniciativas para el negocio. Sin embargo, si intentamos arrancar las 4 al mismo tiempo, los números demuestran que colapsaremos al equipo:

- Nuestra capacidad real y neta del equipo es de 168 horas semanales (descontando el 30% obligatorio que dedicamos a soporte operativo).
- La suma de esfuerzo de las 4 iniciativas es de 610 horas (\$160 + 220 + 140 + 90\$). Si las metemos todas juntas, el 'multitasking' destruirá la productividad, bajará la calidad y disparará los tiempos de entrega por el cuello de botella.
- Además, técnicamente la iniciativa B depende de que la C avance al 50%; iniciar B antes rompería la arquitectura.

Por lo tanto, propongo iniciar ordenadamente con A y C utilizando un modelo WSJF, lo que nos permite trabajar las primeras horas de forma eficiente, cuidar al equipo del burnout y entregar valor real en menos de un mes sin poner en riesgo la operación

4.2 Reporte ejecutivo desde datos de Jira

1. ¿Qué se entregó?

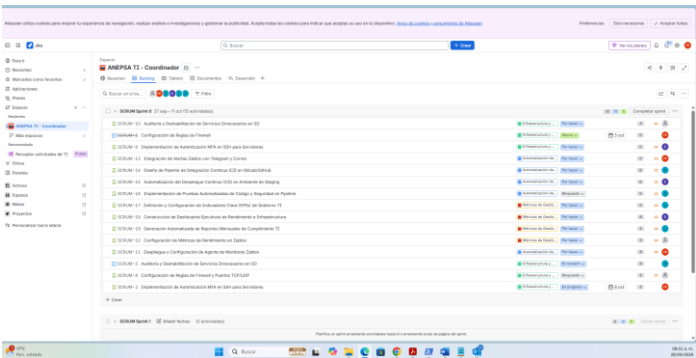
- Concluimos de manera exitosa la Configuración de Reglas de Firewall (SCRUM-1), consolidando la seguridad perimetral de nuestra infraestructura de TI y protegiendo los accesos principales de la organización.

2. ¿Qué está en riesgo?

- Actualmente enfrentamos dos bloqueos operativos que impactan la fluidez del equipo: la *Implementación de Pruebas Automatizadas en Pipeline (SCRUM-16)* y la *Configuración de Reglas de Firewall y Puertos (SCRUM-8)*.
- *Impacto humano:* Estos atorado-recursos generan fricción técnica y frustración en el equipo de desarrollo, poniendo en riesgo el cumplimiento de las metas del sprint si no se liberan a la brevedad.

3. ¿Qué decisión necesitamos de Dirección?

- Solicitamos el apoyo y autorización de la Dirección para destrabar las dependencias técnicas y permisos de red que mantienen detenidas las tareas SCRUM-16 y SCRUM-8, permitiendo que el equipo recupere su ritmo y tranquilidad operativa



4.3 Gestión de stakeholders y proveedores

1. Protocolo de Escalamiento con Proveedor	profesional y colaborativo	Qué: Exigir plan de remediación ante 2 retrasos consecutivos en el módulo SAT. Cuándo: Inmediatamente tras el segundo fallo. Escalamiento: Account manager del proveedor y, de manera interna, Director de TI y Finanzas.
2. Comunicación del Riesgo a Finanzas	Transparente, sin pánico ni minimización	Exponer la situación con datos claros (retraso del proveedor), destacar el impacto real en el cierre de mes, y presentar un plan de mitigación con fechas estimadas de respaldo.

3. Cláusulas y Controles Futuros	Prevención de incertidumbre contractual	Incluir SLAs estrictos con penalizaciones por retrasos consecutivos, entregas parciales obligatorias en ambientes de pruebas (Staging) y juntas de seguimiento quincenales obligatorias.
----------------------------------	---	--

¿Cómo comunicas el riesgo a Finanzas sin generar pánico pero sin minimizarlo:

Buenas tardes equipo de Finanzas. Quería platicarles con total transparencia sobre el estatus de la integración con el SAT para asegurar que su cierre de mes corra sin contratiempos.

El proveedor externo ha acumulado dos retrasos en sus entregas parciales. Esto significa que no tenemos el módulo integrado al 100% en la fecha estimada original como se había programado

Sin embargo, ya activamos un protocolo de escalamiento estricto con su gerencia y estamos evaluando un escenario de respaldo manual o una entrega parcial acelerada para cubrir los requerimientos críticos de su cierre. Les mantendré informados puntualmente cada 48 horas de los avances quedo al pendiente de sus comentarios y sugerencias

Propón cláusulas o controles que deberían existir a futuro con proveedores críticos

1. Acuerdos de Nivel de Servicio (SLA) y Disponibilidad

- Disponibilidad Garantizada (Uptime): Compromiso mínimo de disponibilidad del servicio del 99.9% mensual, excluyendo ventanas de mantenimiento programadas previamente.
- Tiempos de Respuesta ante Incidentes (RTO y RPO): Definición estricta de severidad para fallas críticas (Severidad 1), con un tiempo máximo de respuesta inicial menor a 30 minutos y resolución o mitigación continua en menos de 4 horas.
- Penalizaciones por Incumplimiento: Aplicación de créditos sobre la facturación mensual ante violaciones recurrentes de los SLAs acordados.

2. Seguridad de la Información y Gestión de Vulnerabilidades

- Auditorías y Escaneos de Seguridad: Obligación del proveedor de entregar reportes periódicos de escaneo de vulnerabilidades y pruebas de penetración (Pentest) anuales realizadas por terceros independientes.
- Gestión de Parches de Seguridad: Compromiso de actualizar y aplicar parches críticos de seguridad en un plazo no mayor a 48 horas tras su divulgación oficial (CVE).
- Aislamiento y Protección de Datos: Restricción absoluta en el uso de los datos de la infraestructura o flujos de trabajo corporativos para el entrenamiento de modelos de inteligencia artificial externos sin consentimiento explícito

3. Continuidad Operativa y Gestión de Salida (Exit Strategy)

- Plan de Transición y Reversibilidad: Obligación contractual de facilitar la migración ordenada de datos, configuraciones y código fuente hacia otra plataforma o infraestructura propia en un plazo máximo acordado tras la terminación del contrato.
- Escrow de Código (Depósito en Fianza): En caso de dependencias de software crítico propietario o servicios SaaS cerrados, asegurar el acceso al código fuente o binarios actualizados en un repositorio neutro ante quiebra o cese de operaciones del proveedor.

CASO 5 - SEGURIDAD Y CUMPLIMIENTO

5.1 Checklist de accesos y hardening

Revocación de cuentas inactivas: Verificar y dar de baja permisos de excolaboradores o personal externo temporal.

- Frecuencia de ejecución: cada mes

Auditoría de accesos a Producción: Validar qué usuarios tienen llaves SSH o permisos de acceso directo a los servidores de producción.

- *Frecuencia de ejecución:* Cada mes y medio

Revisión de credenciales en N8N / Herramientas de Automatización: Comprobar que las llaves de API, webhooks y contraseñas estén cifradas.

- *Frecuencia de ejecución:* Cada trimestre (cada 3 meses)

Control de accesos a Repositorios (GitHub/GitLab): Auditar los niveles de permisos (Admin, Maintain, Write, Read) de los desarrolladores.

- *Frecuencia de ejecución:* Cada trimestre (cada 3 meses)

Verificación de Autenticación de Doble Factor (2FA): Asegurar que el 100% de las cuentas con privilegios elevados tengan activo el 2FA.

- *Frecuencia de ejecución:* Cada trimestre (cada 3 meses)

Revisión de políticas de contraseñas: Comprobar vigencia y complejidad robusta en accesos a bases de datos y plataformas críticas.

- *Frecuencia de ejecución:* Cada trimestre (cada 3 meses)

Auditoría de tokens de acceso personal (PAT): Caducar y regenerar tokens de integración que hayan superado su tiempo de vida.

- *Frecuencia de ejecución:* Cada trimestre (cada 3 meses)

Revisión de logs de accesos extraños: Inspeccionar registros en busca de intentos de inicio de sesión fallidos recurrentes.

- *Frecuencia de ejecución:* Mensual (dentro del ciclo trimestral de revisión)

Línea de tiempo de las primeras 2 horas.

Minuto / Ventana	Acción Inmediata de Respuesta	Responsable / Detalle Técnico
Minuto 00 - 15	Revocación de la credencial	Invaldar de inmediato la API key comprometida en la consola del proveedor de pagos para cortar el acceso.
Minuto 15 - 30	Generación de nueva credencial	Crear una nueva llave segura y actualizar las variables de entorno (ENV) o el gestor de secretos de la aplicación.
Minuto 30 - 60	Limpieza del repositorio	Purgar el historial de Git del repositorio público utilizando herramientas especializadas (ej. BFG o git-filter-repo) y forzar actualización (push --force).
Minuto 60 - 90	Auditoría forense de logs	Revisar los registros de actividad del proveedor de pagos de las últimas 6 horas para buscar accesos no autorizados o llamadas sospechosas.
Minuto 90 - 120	Notificación y registro interno	Informar formalmente a Ciberseguridad y Liderazgo Técnico sobre el incidente contenido y documentar el evento.

2. ¿Cómo determinas si la key fue explotada? (Qué revisar exactamente)

Para confirmar si la credencial fue utilizada maliciosamente, se deben auditar las siguientes fuentes:

- **Logs y Dashboards del Proveedor de Pagos:**
 - **Historial de llamadas a la API:** Filtrar por la IP de origen y la credencial comprometida para buscar patrones de tráfico inusuales, peticiones masivas o consultas desde ubicaciones geográficas no reconocidas.
 - **Logs de Autenticación / Errores:** Verificar si se intentaron realizar acciones privilegiadas (creación de reembolsos, transferencias o lectura de datos sensibles de tarjetahabientes).
 - **Registros de Transacciones:** Comprobar si existen cargos, cobros o modificaciones en la pasarela de pagos que no coincidan con la operación legítima de la aplicación en esas 6 horas.
- **Herramientas de Monitoreo del Repositorio:**
 - Revisar si servicios automáticos de escaneo de secretos (como *GitHub Secret Scanning*, *GitGuardian* o *TruffleHog*) detectaron la llave y si reportan alertas de webhooks activados antes de la revocación.

3. ¿A quién notificas internamente y en qué orden? ¿Hay obligación de notificación externa?

Notificación Interna (Orden de prioridad):

1. **Equipo de Seguridad / Ciberseguridad (SOC / CISO):** Para coordinar la respuesta técnica, verificar el alcance del impacto y el monitoreo de la infraestructura.
2. **Líder Técnico / Engineering Manager:** Para informar sobre la afectación en el ciclo de desarrollo y asegurar que el equipo aplique buenas prácticas (evitar *hardcoding* de secretos).
3. **Dirección de Tecnología (CTO) / Dirección General:** Para mantener el gobierno corporativo informado sobre un riesgo crítico de seguridad ya contenido.
4. **Equipo Legal / Cumplimiento:** Para evaluar implicaciones normativas y contractuales.

Obligación de Notificación Externa:

- **Proveedores o Socios de Negocio:** No suele haber una obligación legal estricta de notificación pública si la llave fue revocada en minutos y los logs confirman cero explotaciones. Sin embargo, se debe revisar el acuerdo de nivel de servicio (SLA) o contrato con el proveedor de pagos por políticas específicas de reporte de incidentes.
- **Reguladores o Autoridades de Protección de Datos:** Solo aplica la notificación obligatoria a autoridades y usuarios afectados si el análisis forense demuestra que sí hubo exfiltración de datos financieros sensibles o transacciones no autorizadas que comprometan información personal (PCI-DSS / leyes locales de protección de datos). Si se contuvo a tiempo y sin uso malicioso, el riesgo regulatorio externo suele mitigarse.

Notificación Interna

1. **Equipo de Seguridad / Ciberseguridad (SOC / CISO):** Para coordinar la respuesta técnica, verificar el alcance del impacto y el monitoreo de la infraestructura.
2. **Líder Técnico / Engineering Manager:** Para informar sobre la afectación en el ciclo de desarrollo y asegurar que el equipo aplique buenas prácticas (evitar *hardcoding* de secretos).

3. Dirección de Tecnología (CTO) / Dirección General: Para mantener el gobierno corporativo informado sobre un riesgo crítico de seguridad ya contenido.
4. Equipo Legal / Cumplimiento: Para evaluar implicaciones normativas y contractuales.

Obligación de Notificación Externa:

- Proveedores o Socios de Negocio: No suele haber una obligación legal estricta de notificación pública si la llave fue revocada en minutos y los logs confirman cero explotación. Sin embargo, se debe revisar el acuerdo de nivel de servicio (SLA) o contrato con el proveedor de pagos por políticas específicas de reporte de incidentes.
- Reguladores o Autoridades de Protección de Datos: Solo aplica la notificación obligatoria a autoridades y usuarios afectados si el análisis forense demuestra que sí hubo exfiltración de datos financieros sensibles o transacciones no autorizadas que comprometan información personal (PCI-DSS / leyes locales de protección de datos). Si se contuvo a tiempo y sin uso malicioso, el riesgo regulatorio externo suele mitigarse.

5.3 Cumplimiento

Consideración Regulatoria / Cumplimiento	Impacto en Datos de Clientes y Transacciones	Controles Técnicos de Soporte
1. Privacidad y Protección de Datos (ej. Leyes de Protección de Datos / GDPR / regulaciones locales financieras)	Obligación de proteger la información personal identificable (PII) y los historiales de transacciones de los clientes contra accesos no autorizados.	Cifrado de datos en reposo (AES-256) y en tránsito (TLS 1.3), enmascaramiento de datos sensibles en bases de datos y políticas de retención estricta.
2. Seguridad en Pagos y Estándares Financieros (ej. PCI-DSS)	Garantizar la confidencialidad e integridad de los datos de tarjetas de crédito y cuentas bancarias durante el procesamiento transaccional.	Tokenización de tarjetas, segmentación estricta de redes (VPCs aisladas), escaneos de vulnerabilidades periódicos y control estricto de acceso basado en roles (RBAC).
3. Auditoría y Trazabilidad Operativa (Normativa Contable y Antilavado de Dinero - AML)	Mantener registros inmutables de quién accedió, modificó o autorizó cada transacción y movimiento de datos sensibles de los clientes.	Implementación de pistas de auditoría centralizadas (*Audit Logs*), almacenamiento de logs inmutables protegidos contra borrado y monitoreo automatizado con alertas de seguridad (SIEM).

CASO 6 - GESTIÓN DE CRISIS OPERATIVA

1. Minuto a minuto: Primeros 15 minutos

- Minuto 00:00 - 00:03 **Detección y Congelamiento:** Al recibir la alerta a las 7:50 AM, lo primero es **bloquear el acceso a la base de datos de producción** para evitar que la corrupción de la tabla de transacciones siga propagándose.
- Minuto 00:03 - 00:07 **Evaluación Humana Inmediata:** Localizo al desarrollador y le digo ocurrió un incidente con la migración. Tranquilo, vamos a enfocarnos en mitigar la afectación lo antes posible
- Minuto 00:07 - 00:12 **(Aislamiento de la Evidencia del Sistema):** Ejecuto una consulta rápida de diagnóstico en modo solo lectura para determinar el volumen de registros afectados y el último estado íntegro antes del script erróneo.
- Minuto 00:12 - 00:15 **(Toma de Decisión de Ruta Crítica):** Con base en el estado del sistema, decido si es viable un parche o si activamos el protocolo de restauración completa, informando al equipo técnico principal la situación

2. Evaluación técnica del alcance del daño

Antes de decidir si restauramos el respaldo de hace 26 horas o intentamos una reparación parcial, ejecuto los siguientes pasos analíticos:

- **Análisis del Log de Transacciones (WAL / Binlog):** Reviso los registros de transacciones para identificar exactamente qué comandos SQL ejecutó el script (por ejemplo, si hizo un UPDATE masivo sin WHERE o un DELETE lógico/físico corrupto).

- **Cuantificación del Δ (Delta) de Datos:** Mido la diferencia exacta de transacciones perdidas si aplicamos el respaldo de hace 26 horas frente al estado actual corrompido:
 - **Respaldo de 26 horas:** Garantiza integridad estructural al 100%, pero pierde un día completo de operaciones recientes.
 - **Reparación Parcial:** Viable solo si la corrupción está acotada
- **Decisión:** Si la corrupción rompió las restricciones de integridad referencial (Foreign Keys) o hay datos anómalos imposibles de trazar de forma segura en menos de 30 minutos, la regla de oro es restaurar el backup de hace 26 horas a un entorno aislado/secundario para rescatar mediante scripts de *diff* únicamente las transacciones legítimas ocurridas en las últimas horas y volcarlas de forma limpia.

4. **Guión de comunicación al equipo de Finanzas**

Primer mensaje Inmediato - 8:00 AM, antes de tener la solución

- Buenos días equipo de Finanzas. Les informamos que estamos atendiendo una incidencia técnica crítica en la base de datos de producción detectada hace pocos minutos. Nuestro equipo de ingeniería está trabajando en la recuperación y validación de los datos. Les mantendremos informados y daremos una nueva actualización a más tardar a las 8:30 AM para confirmar el estatus de su corte de las 9:00 AM. Agradecemos su comprensión

segundo mensaje Una hora después - 8:55 AM

- Actualización sobre el corte de Finanzas: Se aplicó nuestro protocolo de recuperación de datos. La integridad estructural ha sido restablecida mediante nuestro respaldo más reciente y un re-proceso controlado de transacciones. El sistema está operativo y los datos para el corte de las 9:00 AM están listos y validados. Ofrecemos una disculpa por el inconveniente y estamos implementando controles adicionales para que no vuelva a ocurrir.

Conversación individual posterior con el desarrollador Junior

Primero que nada, qué bueno que hablemos más tranquilos. Quiero que sepas que tu empleo no está en riesgo y entiendo perfectamente el nivel de estrés por el que pasaste al ver que el script afectó producción. Todos los ingenieros seniors hemos cometido errores graves en algún momento de nuestras carreras.

El problema real aquí no fue tu error humano; el error fue estructural y de procesos, ya que el sistema jamás debió permitir que una credencial de desarrollo o un script de migración tuviera acceso directo de escritura (DDL/DML destructivo) a la base de datos de producción sin una doble validación o una ejecución controlada por pipeline.

Veamos esto como una oportunidad de aprendizaje: en el post-mortem blameless de esta semana documentaremos qué faltó en los permisos y en las herramientas para blindar el flujo. Tu tarea ahora no es cargar con la culpa, sino ayudarnos a diseñar los candados técnicos para que ningún otro desarrollador pueda volver a ejecutar esto por accidente. ¿Te parece bien?

Tres controles técnicos concretos para prevenirlo

Segregación Estricta de Credenciales y Roles (IAM / RBAC): Las cuentas o tokens utilizados por los desarrolladores para el entorno de *staging* o desarrollo local deben tener denegación explícita (Deny Policy) en las bases de datos de producción a nivel de red y base de datos. Producción solo es modificable mediante pipelines automatizados de CI/CD.

Protección contra Migraciones Directas en CLI: Configurar los ORM o herramientas de migración (como ActiveRecord, Prisma o Flyway) para que rechacen la ejecución de comandos de migración (db:migrate, etc.) si la variable de entorno `RAILS_ENV / NODE_ENV` apunta a `production` y no se ejecuta desde un runner autorizado de GitHub Actions / GitLab CI.

Implementación de Puntos de Salva Automáticos y Prevención de Concurrencia: Habilitar *Point-in-Time Recovery (PITR)* automatizado en el proveedor de bases de datos para reducir la ventana de respaldo de 26 horas a menos de 5 minutos, y añadir un script de validación previa (*dry-run*) obligatorio que bloquee scripts con operaciones destructivas masivas (DROP, TRUNCATE, UPDATE sin WHERE) en entornos productivos.

CASO 7 - PLANEACIÓN ESTRATÉGICA, IA Y KPIS:

7.1 Auditoría de deuda técnica y plan 30-60-90 días

auditoría en las primeras 2 semanas

Herramientas a utilizar:

- Jira / Tablero de gestión: Para extraer la lista completa de tickets y su antigüedad.
- Inventario de N8N: Revisión directa en la plataforma de automatización para listar los 18 workflows existentes.
- Git / Repositorio de código: Para auditar ramas, despliegues y verificar la ausencia de entornos aislados (dev/QA/prod).

Criterio de Auditoría:

- Clasificar los tickets por impacto al negocio
- Evaluar el riesgo operativo de las 18 automatizaciones de N8N ¿qué pasa si fallan? ¿quién sabe cómo funcionan?

Fórmula simple para cuantificar la deuda técnica en horas y costo:

- *Fórmula: Deuda Total (Horas) = Tickets sin criterio x Horas promedio de análisis + Workflows N8N sin documentar x Horas de ingeniería inversa*
- *Costo Financiero: $\text{Deuda Total (\$)} = \text{Deuda Total (Horas)} \times \text{Costo por hora del recurso técnico (Salario/Carga prestacional)}$*
- *Ejemplo: Si estimamos 2 horas por cada uno de los 60 tickets (\$120) y 4 horas por cada uno de los 18 workflows sin documentar (\$72), tenemos \$192 horas totales de deuda técnica acumulada que el equipo está pagando en ineficiencia mensual.*

Matriz de criticidad para las 18 automatizaciones N8N

Automatización N8N	Impacto Financiero / Operativo (1-5)	Frecuencia de Fallo (1-5)	Dependencia de una sola persona (1-5)	Complejidad de Reconstrucción (1-5)	Puntaje de Criticidad Total
Flujo 01 (Ej. Sincronización de Pagos / ERP)	5 (Crítico para ingresos)	4 (Falla semanalmente)	5 (Solo lo entiende el ex-colaborador)	4 (Alto acoplamiento con APIs)	18 / 20 (Crítico - Prioridad 1)
Flujo 02 (Ej. Notificaciones de Clientes)	2 (Bajo impacto comercial)	1 (Estable)	3 (Documentación parcial)	2 (Bajo)	8 / 20 (Bajo - Prioridad 3)

Plan 30-60-90 días:

Primeros 30 días (Quick Wins obligatorios y reales):

Primeros 30 días (Quick Wins obligatorios y reales):

1. *Establecer la Definición de Listo (DoD):* Prohibir la entrada de tickets a desarrollo en Jira si no cuentan con al menos una descripción clara y criterios de aceptación básicos.
2. *Creación de un entorno básico de Staging/QA:* Aislar un ambiente mínimo de pruebas para que los desarrolladores no desplieguen directamente sin validar.
3. *Documentación relámpago de N8N:* Realizar sesiones de grabación de 15 minutos por cada flujo crítico de N8N con el equipo para mitigar el riesgo de dependencia individual.

Día 60 (Consolidación y Capacitación del Líder Técnico):

- Liberar de sobrecarga operativa al líder técnico asignando tareas de mentoría a los 4 desarrolladores Full Stack y automatizando la asignación de tareas en Jira. Reducir el tiempo promedio de entrega de 35 a 25 días.

Día 90 (Cambio Estructural):

- Implementación de un pipeline de CI/CD formal con separación estricta de entornos (Dev, QA, Prod), reduciendo los tiempos de entrega por debajo de los 15 días y estableciendo gobernanza basada en KPIs de entrega continua.

7.2 Tres automatizaciones prioritarias con IA

1. Automatización 1: Procesamiento y Clasificación de Facturas / Comprobantes PDF

- Problema operativo cuantificado: El equipo administrativo gasta 10 horas semanales leyendo manualmente PDFs de facturas o comprobantes de proveedores para extraer conceptos, fechas y montos e ingresarlos al sistema.
- Stack técnico exacto:
 - *Nodos N8N:* Webhook (recepción de archivo) -> HTTP Request (envío del PDF al endpoint del modelo de IA con prompt estructurado) -> PostgreSQL Node (inserción de datos estructurados).
 - *Endpoint del Modelo:* https://api.openai.com/v1/chat/completions (usando GPT-4o-mini con capacidad multimodal de lectura de documentos) o Google Gemini API (Gemini Flash).
 - *Tablas PostgreSQL:* invoices (campos: id, provider_name, amount, tax_date, status).
- Manejo de errores: Si el modelo de IA devuelve un JSON malformado o falla la conexión, N8N desvía el flujo mediante un nodo Error Trigger hacia una tabla de respaldo invoice_failures y envía una alerta a un canal de Slack/correo para revisión.
- KPI de éxito: Reducción del tiempo de procesamiento manual de 10 horas a menos de 1 hora semanal (90% de reducción en la tarea).

2. Automatización 2: Extracción y Clasificación Automática de Tickets de Soporte en Jira / Mesa de Ayuda

- Problema operativo cuantificado: El equipo pierde 8 horas semanales leyendo manualmente reportes y quejas de usuarios para categorizarlas, estimar prioridad y asignarlas al área correcta.
- Stack técnico exacto:

- **Nodos N8N:** Jira Trigger (nuevo ticket creado sin clasificar) -> OpenAI / Gemini API Node (análisis de texto y clasificación de prioridad/categoría) -> Jira Update Node (actualización automática de etiquetas y campos).
- **Endpoint del Modelo:** https://api.openai.com/v1/chat/completions o endpoint equivalente de Gemini para clasificación de texto natural (text-classification).
- **Tablas PostgreSQL (Auditoría interna):** ticket_ai_logs (campos: ticket_id, predicted_category, confidence_score, processed_at).
- **Manejo de errores:** Si el ticket contiene texto en blanco o caracteres extraños que rompen el prompt, se le asigna una categoría por defecto Revisión Manual Requerida y se notifica al coordinador sin detener el flujo de Jira.
- **KPI de éxito:** Reducción del tiempo de clasificación inicial de 8 horas a 0 horas humanas (100% automatizado) y disminución del tiempo de respuesta al cliente.

3. Automatización 3: Detección y Resumen de Anomalías en Logs de Transacciones

- **Problema operativo cuantificado:** Los analistas invierten 7 horas semanales revisando manualmente bitácoras de errores de la base de datos para identificar patrones repetitivos.
- **Stack técnico exacto:**
 - **Nodos N8N:** Schedule Trigger (ejecución cada hora) -> PostgreSQL Node (consulta de logs de la última hora con errores) -> OpenAI / Gemini API Node -> Email / Telegram Node envío de reporte al equipo técnico.
 - **Endpoint del Modelo:** https://api.openai.com/v1/chat/completions (para síntesis de logs).
 - **Tablas PostgreSQL:** system_logs y ai_anomaly_reports.
- **Manejo de errores:** Si la consulta a PostgreSQL no arroja resultados (cero errores en la hora), N8N intercepta el arreglo vacío mediante un nodo If y finaliza el flujo silenciosamente sin mandar alertas innecesarias.
- **KPI de éxito:** Reducción de 7 horas semanales de revisión manual de logs a una lectura de resumen ejecutivo de 10 minutos.

Bonus: ¿Cómo usar un agente de IA para detectar anomalías o duplicados antes de que un dato entre a producción, sin bloquear el flujo normal si el modelo falla o tarda?

Para lograr esto sin arriesgar la disponibilidad ni bloquear la operación si la API de IA se cae o responde lento, implementamos un flujo asíncrono y de doble capa:

1. **Validación Estricta Nativa (Capa 1 - Síncrona e Instantánea):** El sistema aplica reglas duras en la base de datos (restricciones de unicidad UNIQUE, llaves primarias y validaciones de formato en PostgreSQL). Esto garantiza que, si la IA tarda o falla, el flujo normal nunca se detiene ni se bloquea.
2. **Validación Inteligente por IA en Segundo Plano (Capa 2 - Asíncrona):**
 - El dato entra a producción de forma limpia tras pasar la Capa 1.
 - Mediante un Webhook o una cola en N8N, se dispara una consulta asíncrona enviando el registro al modelo de IA para buscar similitudes semánticas complejas o patrones anómalos que las reglas duras no detectan (ej. un cliente registrado con un nombre ligeramente alterado pero misma dirección).
 - Si la IA detecta un posible duplicado o anomalía, no bloquea la inserción previa, sino que inserta una alerta en una tabla de revisión (fraud_or_duplicate_flags) para que el equipo operativo lo revise en su panel sin afectar la experiencia del usuario final.
3. **Manejo de Fallos (Timeouts / Caídas de API):** Configurar un límite de tiempo estricto (timeout de 3 segundos) en el nodo HTTP de N8N hacia la IA. Si la API de OpenAI/Gemini no responde en ese lapso, el flujo captura el error mediante un bloque de excepciones (Catch Error), lo registra internamente como "IA No Disponible" y permite que la operación continúe su curso habitual sin impactar al negocio.

7.3 Framework de KPIs para gobernar el equipo

Tabla de KPIs (6 KPIs distribuidos en 3 dimensiones)

Dimensión	KPI	Fórmula	Frecuencia	Umbral de alerta
Productividad	Velocity (Puntos de Historia completados)	Suma de Story Points de tickets cerrados en el Sprint	Por sprint (quincenal)	< 80% de la velocidad promedio histórica
Productividad	Throughput (Tickets entregados)	Conteo total de tickets movidos a "Done" por sprint	Por sprint (quincenal)	Caída mayor al 30% respecto al sprint anterior
Calidad	Tasa de Defectos en Producción (Bugs)	(Número de bugs reportados en producción / Total de despliegues) * 100	Mensual	> 5% de incidencias sobre despliegues
Calidad	Densidad de Deuda Técnica	Tickets de deuda técnica resueltos vs. tickets nuevos acumulados en el periodo	Mensual	Crecimiento neto positivo de deuda técnica en 2 sprints seguidos
Estabilidad / Operación	Lead Time (Tiempo de entrega)	Fecha de cierre del ticket - Fecha de inicio del desarrollo	Por sprint	> 15 días promedio por ticket
Estabilidad / Operación	MTTR (Tiempo Medio de Recuperación)	Suma total de tiempo de inactividad / Número de incidentes críticos resueltos	Mensual / Trimestral	> 2 horas por incidente crítico

2. Presentación de KPIs

- Nombre presentación: *Sprint Review y Gobernanza Operativa de TI.*
- Participantes: Coordinador de TI (tú), Líder Técnico y los 4 Desarrolladores Full Stack.
- Duración: 45 minutos quincenales (al cierre de cada sprint).
- Agenda:
 1. Revisión de tableros y cumplimiento de los 6 KPIs (15 min).
 2. Análisis de cuellos de botella y bloqueos operativos (15 min).
 3. Alineación de prioridades y compromisos para el siguiente ciclo (15 min).
- Salida: Minuta corta en Jira y actualización del tablero de métricas del equipo.

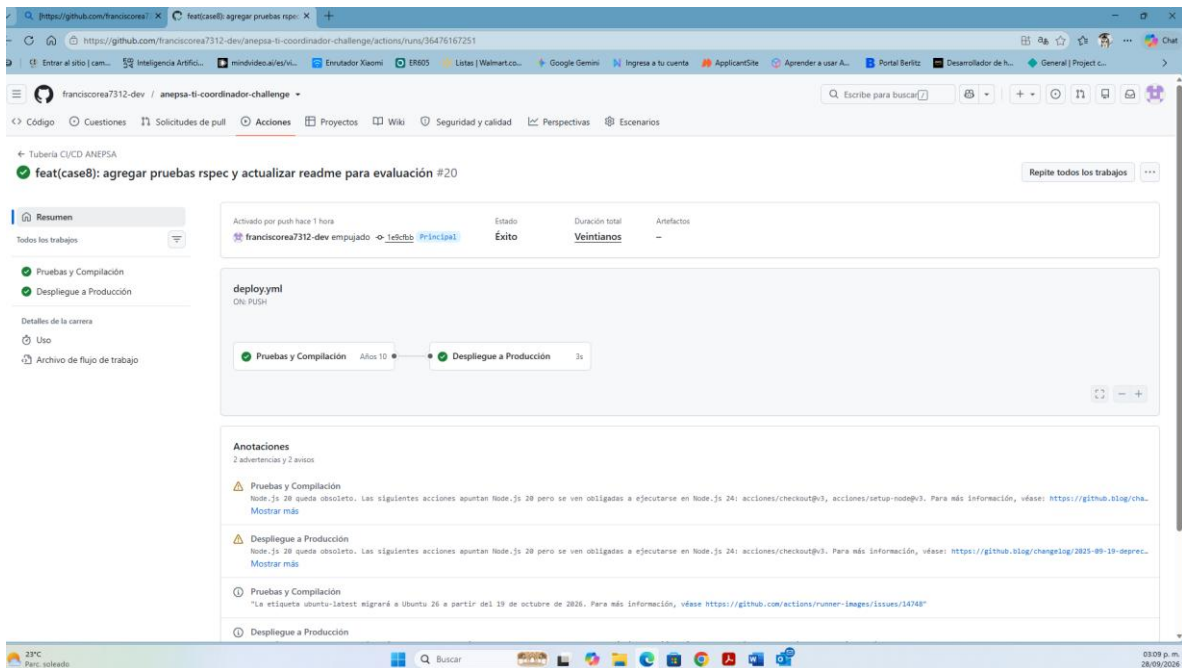
3. ¿Qué hacer si el equipo está "maquillando" un KPI cerrando tickets sin resolver

- ¿Cómo se detecta? (Señales de alerta):
 - *Discrepancia de Calidad:* El KPI de "Velocity" o "Throughput" se ve excelente (muchos tickets cerrados), pero el KPI de "Defectos en Producción" o la cantidad de reportes de soporte se dispara de inmediato.
 - *Inspección de Criterios:* Al revisar aleatoriamente tickets cerrados en Jira, notas que no tienen pruebas adjuntas, los comentarios de revisión están vacíos o el código asociado no cumple con los criterios de aceptación acordados.
- ¿Qué hacer ejecutivamente?

- **Evitar el juicio punitivo:** No culpar directamente al desarrollador, ya que el maquillaje de métricas suele ser síntoma de miedo a la presión por los números o metas irreales.
- **Endurecer la Definición de Listo (DoD):** Auditar el proceso y establecer que un ticket no se considera cerrado si no pasa por una revisión cruzada de código y validación en un ambiente de pruebas.
- **Redirigir el enfoque de gestión:** Dejar claro en el equipo que el objetivo del KPI no es alcanzar un número arbitrario para impresionar a la dirección, sino medir el flujo real de valor para estabilizar la operación.

[franciscorea7312-dev/anepea-ti-coordinador-challenge](https://github.com/franciscorea7312-dev/anepea-ti-coordinador-challenge)

8.2 Automatización



GitHub Actions workflow for CI/CD pipeline. The workflow is titled "feat(case8): agregar pruebas rspec y actualizar readme para evaluación #20". It shows a "deploy.yml" file with a "ON PUSH" trigger. The workflow consists of two steps: "Pruebas y Compilación" (Tests and Compilation) and "Despliegue a Producción" (Deployment to Production). The "Pruebas y Compilación" step is marked as successful. The "Despliegue a Producción" step is also marked as successful. The workflow is activated by a push event. The status is "Éxito" (Success). The duration is "Veintianos" (Twenty). The workflow is located in the "anepea-ti-coordinador-challenge" repository.

8.3 Demostración funcional

URL del repositorio: <https://github.com/franciscorea7312-dev/anepea-ti-coordinador-challenge>